CREW App — Test Data Seed Tool: Instructions for Lovable.dev

Overview

Create a developer-only seed tool inside the admin panel that generates 50 realistic test users, books them onto an upcoming run, and allows full teardown. This is used exclusively to test the automated group matching flow end-to-end.

⚠️ GUARDRAILS — READ FIRST

DO NOT modify any of the following:

- Any existing Supabase migration files
- Any existing edge functions
- Any auth, booking, onboarding, or user-facing pages
- `src/integrations/supabase/client.ts`
- `src/lib/`, `src/components/ui/`, `src/components/PageShell.tsx`
- `src/components/BottomNav.tsx`
- `src/App.tsx`

Permitted modifications:

- `src/pages/admin/Groups.tsx` — add a collapsible "Developer Tools" section at the very bottom only

New files to create:

- `src/hooks/use-seed-test-data.ts` — all seed/clear logic lives here, keeping Groups.tsx clean

---

Step 1 — Read Existing Interests First

Before writing any code, open `src/pages/Onboarding.tsx` and locate the array of interest options presented to users during onboarding (they will be rendered as pill/chip options). Extract the exact string values used. These are the only interests that should be used in the seed data, so that test users have interests that are compatible with the matching algorithm.

Use those exact strings in the `INTERESTS` constant in Step 2. Do not invent new interest labels.

Step 2 — New Hook: `src/hooks/use-seed-test-data.ts`

Create this file containing all seed and clear logic. It exports two async functions consumed by the UI in Step 3.

typescript

```
import { supabase } from "@/integrations/supabase/client";

// ── Constants ──────────────────────────────────────────────────────────
// IMPORTANT: Replace this array with the exact interest strings
// found in src/pages/Onboarding.tsx before implementing.
const INTERESTS: string[] = [
  // e.g. "Music", "Cycling", "Travel" — copy verbatim from Onboarding.tsx
];

const TEST_EMAIL_DOMAIN = "crew-test.dev";
const TEST_EMAIL_PREFIX = "testuser";

// 50 realistic UK first + last name combinations
const TEST_NAMES: string[] = [
  "Alex Morgan", "Jamie Clarke", "Sam Patel", "Jordan Ellis", "Casey Brooks",
  "Morgan Davies", "Riley Shah", "Taylor Bennett", "Avery Khan", "Reece Cooper",
  "Frankie Hughes", "Billie Okafor", "Charlie Foster", "Stevie Nwosu", "Robbie Patel",
  "Ash Griffiths", "Quinn Mensah", "Elliot Osei", "Darcy Walsh", "Hayden Byrne",
  "Blake Nkosi", "Emerson Diallo", "Addison Clarke", "Remy Okonkwo", "Sage Thompson",
  "Logan Adeyemi", "Finley Olu", "River Bannister", "Sasha Obi", "Drew Kamara",
  "Cody Mensah", "Kai Abara", "Jude Lawson", "Skye Oduya", "Ellis Musa",
  "Rowan Amara", "Harley Ejike", "Flynn Achebe", "Marlowe Eze", "Luca Asante",
  "Nico Babatunde", "Tate Chukwu", "Zara Osei", "Bryn Nwofor", "Orla Kalu",
  "Beau Ibe", "Wynne Okolie", "Rory Anyanwu", "Piper Nwosu", "Scout Obi"
];

// ── Helpers ────────────────────────────────────────────────────────────

function randomDOB(): string {
  // Ages 18–40: birth years between (current year - 40) and (current year - 18)
  const currentYear = new Date().getFullYear();
  const year = currentYear - 18 - Math.floor(Math.random() * 23); // 18 to 40
  const month = String(Math.floor(Math.random() * 12) + 1).padStart(2, "0");
  const day = String(Math.floor(Math.random() * 28) + 1).padStart(2, "0");
  return `${year}-${month}-${day}`;
}

function randomInterests(): string[] {
  // Pick 2–4 interests at random per user for realistic variance
  const shuffled = [...INTERESTS].sort(() => Math.random() - 0.5);
  const count = 2 + Math.floor(Math.random() * 3); // 2, 3, or 4
  return shuffled.slice(0, count);
}

function testEmail(index: number): string {
  return `${TEST_EMAIL_PREFIX}+${index}@${TEST_EMAIL_DOMAIN}`;
```

```
}

// ── Seed Function ──────────────────────────────────────────────────────────

export async function seedTestData(
  onProgress: (message: string) => void
): Promise<{ success: boolean; error?: string; runDateId?: string }> {
  try {
    // 1. Find or create a run_date ~47 hours from now (inside the matching window)
    onProgress("Finding upcoming run date…");

    const now = new Date();
    const targetTime = new Date(now.getTime() + 47 * 60 * 60 * 1000);
    const targetDate = targetTime.toISOString().split("T")[0];
    const targetTimeStr = targetTime.toTimeString().slice(0, 8); // HH:MM:SS

    let runDateId: string;

    const { data: existingRunDate } = await supabase
      .from("run_dates")
      .select("id, date")
      .gte("date", new Date().toISOString().split("T")[0])
      .order("date", { ascending: true })
      .limit(1)
      .maybeSingle();

    if (existingRunDate) {
      runDateId = existingRunDate.id;
      onProgress(`Using existing run date: ${existingRunDate.date}`);
    } else {
      // Create a run date 47h from now — this requires admin/service access.
      // If insert fails due to RLS, instruct the user to create a run date manually first.
      const { data: newRunDate, error: rdErr } = await supabase
        .from("run_dates")
        .insert({
          date: targetDate,
          time: targetTimeStr,
          meeting_point: "The Bandstand, Battersea Park",
          price_pence: 1200,
        })
        .select("id")
        .single();

      if (rdErr) {
        return {
          success: false,
          error: `No upcoming run date found and could not create one automatically. Please create a run date manually in the adm
```

```
      };
    }

    runDateId = newRunDate.id;
    onProgress(`Created new run date: ${targetDate} at ${targetTimeStr}`);
  }

  // 2. Create 50 test users via Supabase Auth Admin API
  // Note: supabase.auth.admin.createUser requires the service role key.
  // In Lovable, use supabase.functions.invoke to call a seed edge function
  // (see Step 3 below) rather than calling admin auth directly from the browser.

  onProgress("Creating 50 test users…");

  const { data: seedResult, error: seedErr } = await supabase.functions.invoke(
    "seed-test-data",
    {
      body: { run_date_id: runDateId, names: TEST_NAMES },
    }
  );

  if (seedErr || seedResult?.error) {
    return {
      success: false,
      error: seedErr?.message ?? seedResult?.error,
    };
  }

  onProgress(`✓ ${seedResult.users_created} test users created and booked.`);
  return { success: true, runDateId };
 } catch (err: any) {
  return { success: false, error: err.message };
 }
}

// ── Clear Function ──────────────────────────────────────────────────────

export async function clearTestData(
 onProgress: (message: string) => void
): Promise<{ success: boolean; error?: string }> {
 try {
  onProgress("Clearing test data…");

  const { data: result, error } = await supabase.functions.invoke(
    "seed-test-data",
    { body: { action: "clear" } }
  );
```

```
    if (error || result?.error) {
      return { success: false, error: error?.message ?? result?.error };
    }

    onProgress(`✓ Removed ${result.users_deleted} test users and all related records.`);
    return { success: true };
  } catch (err: any) {
    return { success: false, error: err.message };
  }
}
```

Step 3 — New Edge Function: `supabase/functions/seed-test-data/index.ts`

All auth-admin operations (creating users, deleting users) require the service role key and must run server-side. This edge function handles both seeding and clearing.

```typescript
```

```
import { serve } from "https://deno.land/std@0.190.0/http/server.ts";
import { createClient } from "npm:@supabase/supabase-js@2.57.2";

const corsHeaders = {
  "Access-Control-Allow-Origin": "*",
  "Access-Control-Allow-Headers":
    "authorization, x-client-info, apikey, content-type",
};

const TEST_EMAIL_DOMAIN = "crew-test.dev";
const TEST_EMAIL_PREFIX = "testuser";
const TEST_PASSWORD = "CrewTest2026!"; // uniform password for all test accounts

// ── Helpers ──────────────────────────────────────────────────────────────

function randomDOB(): string {
  const currentYear = new Date().getFullYear();
  const year = currentYear - 18 - Math.floor(Math.random() * 23); // ages 18–40
  const month = String(Math.floor(Math.random() * 12) + 1).padStart(2, "0");
  const day = String(Math.floor(Math.random() * 28) + 1).padStart(2, "0");
  return `${year}-${month}-${day}`;
}

function randomInterests(allInterests: string[]): string[] {
  const shuffled = [...allInterests].sort(() => Math.random() - 0.5);
  const count = 2 + Math.floor(Math.random() * 3); // 2–4 interests
  return shuffled.slice(0, count);
}

function testEmail(index: number): string {
  return `${TEST_EMAIL_PREFIX}+${index}@${TEST_EMAIL_DOMAIN}`;
}

// ── Handler ──────────────────────────────────────────────────────────────

serve(async (req) => {
  if (req.method === "OPTIONS") {
    return new Response(null, { headers: corsHeaders });
  }

  const supabaseAdmin = createClient(
    Deno.env.get("SUPABASE_URL") ?? "",
    Deno.env.get("SUPABASE_SERVICE_ROLE_KEY") ?? ""
  );

  const respond = (body: object, status = 200) =>
```

```
  new Response(JSON.stringify(body), {
    headers: { ...corsHeaders, "Content-Type": "application/json" },
    status,
  });

try {
  // Validate caller is admin
  const authHeader = req.headers.get("Authorization");
  if (!authHeader) throw new Error("No authorization header");
  const token = authHeader.replace("Bearer ", "");
  const { data: authData, error: authErr } = await supabaseAdmin.auth.getUser(token);
  if (authErr || !authData.user) throw new Error("Not authenticated");

  const { data: roleRow } = await supabaseAdmin
    .from("user_roles")
    .select("role")
    .eq("user_id", authData.user.id)
    .eq("role", "admin")
    .maybeSingle();
  if (!roleRow) throw new Error("Forbidden: admin only");

  // Parse body
  let body: any = {};
  try { body = await req.json(); } catch { /* empty body */ }

  // ── CLEAR ACTION ──────────────────────────────────────────────
  if (body.action === "clear") {
    // Find all test user accounts by email pattern
    // Note: Supabase admin.listUsers is paginated; fetch all pages
    let allUsers: any[] = [];
    let page = 1;
    while (true) {
      const { data: { users }, error } = await supabaseAdmin.auth.admin.listUsers({
        page,
        perPage: 1000,
      });
      if (error) throw error;
      allUsers = allUsers.concat(users);
      if (users.length < 1000) break;
      page++;
    }

    const testUsers = allUsers.filter((u) =>
      u.email?.endsWith(`@${TEST_EMAIL_DOMAIN}`)
    );

    if (testUsers.length === 0) {
```

```
  return respond({ success: true, users_deleted: 0, message: "No test users found." });
}

const testUserIds = testUsers.map((u) => u.id);

// Delete in dependency order:
// group_messages → run_group_members → run_groups → bookings → profiles → auth users

// 1. Find run groups that contain only test users (to avoid touching real data)
//    Simpler: delete group_messages authored by test users
await supabaseAdmin
  .from("group_messages")
  .delete()
  .in("user_id", testUserIds);

// 2. Remove test users from any group memberships
await supabaseAdmin
  .from("run_group_members")
  .delete()
  .in("user_id", testUserIds);

// 3. Delete notifications for test users
await supabaseAdmin
  .from("notifications")
  .delete()
  .in("user_id", testUserIds);

// 4. Delete bookings for test users
await supabaseAdmin
  .from("bookings")
  .delete()
  .in("user_id", testUserIds);

// 5. Delete profiles
await supabaseAdmin
  .from("profiles")
  .delete()
  .in("id", testUserIds);

// 6. Delete auth users
for (const userId of testUserIds) {
  await supabaseAdmin.auth.admin.deleteUser(userId);
}

// 7. Clean up any run_groups that now have zero members
//    (groups created purely by the seeder)
const { data: emptyGroups } = await supabaseAdmin
```

```
    .from("run_groups")
    .select(`
      id,
      run_group_members ( id )
    `)
    .eq("status", "pending");

  const emptyGroupIds = (emptyGroups ?? [])
    .filter((g: any) => g.run_group_members.length === 0)
    .map((g: any) => g.id);

  if (emptyGroupIds.length > 0) {
    await supabaseAdmin
      .from("run_groups")
      .delete()
      .in("id", emptyGroupIds);
  }

  return respond({ success: true, users_deleted: testUserIds.length });
}

// ── SEED ACTION (default) ──────────────────────────────────────────
const { run_date_id, names } = body;
if (!run_date_id) throw new Error("run_date_id is required");
if (!Array.isArray(names) || names.length === 0) throw new Error("names array is required");

// Read interests from the database — fetch a sample of real profiles
// to extract the interest options actually used in the app.
// Fall back to a hardcoded list if profiles table is empty.
// IMPORTANT: Lovable must replace this fallback array with the exact
// interest strings from src/pages/Onboarding.tsx.
const INTERESTS_FALLBACK: string[] = [
  // Copy exact interest strings from src/pages/Onboarding.tsx here
];

// Check for existing test users to avoid duplicates
const { data: existingTestBookings } = await supabaseAdmin
  .from("bookings")
  .select("user_id, profiles:user_id ( id )")
  .eq("run_date_id", run_date_id);

const existingProfileIds = new Set(
  (existingTestBookings ?? []).map((b: any) => b.user_id)
);

let usersCreated = 0;
const errors: string[] = [];
```

```
for (let i = 0; i < names.length; i++) {
  const email = testEmail(i + 1);
  const fullName = names[i];

  try {
    // Skip if this test user already exists for this run date
    // (check by email)
    const { data: existingUser } = await supabaseAdmin.auth.admin
      .listUsers({ page: 1, perPage: 1000 });
    const alreadyExists = existingUser.users.find((u) => u.email === email);

    let userId: string;

    if (alreadyExists) {
      userId = alreadyExists.id;
    } else {
      // Create auth user
      const { data: newUser, error: createErr } =
        await supabaseAdmin.auth.admin.createUser({
          email,
          password: TEST_PASSWORD,
          email_confirm: true, // skip email verification
          user_metadata: { full_name: fullName },
        });

      if (createErr) {
        errors.push(`User ${i + 1}: ${createErr.message}`);
        continue;
      }

      userId = newUser.user.id;

      // Upsert profile with test data
      await supabaseAdmin.from("profiles").upsert({
        id: userId,
        full_name: fullName,
        date_of_birth: randomDOB(),
        personality_answers: {
          interests: randomInterests(INTERESTS_FALLBACK),
        },
      });
    }

    // Create confirmed booking if one doesn't already exist
    if (!existingProfileIds.has(userId)) {
      const { error: bookingErr } = await supabaseAdmin
```

```
          .from("bookings")
          .insert({
            user_id: userId,
            run_date_id,
            status: "confirmed",
            // stripe_payment_intent_id left null — test bookings bypass Stripe
          });

        if (bookingErr && !bookingErr.message.includes("duplicate")) {
          errors.push(`Booking for ${fullName}: ${bookingErr.message}`);
          continue;
        }
      }

      usersCreated++;
    } catch (err: any) {
      errors.push(`User ${i + 1}: ${err.message}`);
    }
  }

  return respond({
    success: true,
    users_created: usersCreated,
    errors: errors.length > 0 ? errors : undefined,
  });
} catch (err: any) {
  return respond({ error: err.message }, 500);
}
});
```

Add to `supabase/config.toml`:

```toml
# seed-test-data is admin-only — JWT verified inside the function.
[functions.seed-test-data]
verify_jwt = true
```

Step 4 — Update `src/pages/admin/Groups.tsx`

Add a collapsible "Developer Tools" section at the **very bottom** of the Groups component's JSX, after the groups grid. Do not touch any other part of the file.

Import the two functions from the hook at the top of the file:

```tsx
import { seedTestData, clearTestData } from "@/hooks/use-seed-test-data";
```

Then add this state and JSX inside the `Groups` component:

State to add (inside the component, with existing state declarations):

```tsx
const [devToolsOpen, setDevToolsOpen] = useState(false);
const [seedLog, setSeedLog] = useState<string[]>([]);
const [seedRunning, setSeedRunning] = useState(false);
const [clearRunning, setClearRunning] = useState(false);
```

JSX to append at the bottom of the returned JSX (after the groups grid, before the closing `</div>`):

```tsx
```

```
{/* ── Developer Tools ──────────────────────────────────────────────── */}
<div className="mt-10 border-t border-dashed border-border pt-6">
  <button
    type="button"
    onClick={() => setDevToolsOpen((v) => !v)}
    className="flex items-center gap-2 text-xs text-muted-foreground hover:text-foreground transition-colors"
  >
    <span className="font-mono">{devToolsOpen ? "▾" : "▸"}</span>
    Developer Tools
  </button>

  {devToolsOpen && (
    <div className="mt-4 rounded-sm border border-dashed border-destructive/40 bg-destructive/5 p-5 space-y-4">
      <div>
        <p className="text-xs font-semibold text-destructive uppercase tracking-wider">
          ⚠ Dev Only — Do Not Use in Production
        </p>
        <p className="text-xs text-muted-foreground mt-1">
          Generates 50 test accounts (aged 18–40) and books them onto the next
          upcoming run date. Test accounts use emails ending in{" "}
          <code className="font-mono">@crew-test.dev</code> and can be fully
          removed with "Clear Test Data". A run date must exist before seeding.
        </p>
      </div>

      {/* Run Date Requirement Notice */}
      {!activeRunDate && (
        <p className="text-xs text-amber-600 bg-amber-50 border border-amber-200 rounded px-3 py-2">
          Select a run date above before seeding, or the seeder will use the
          next available upcoming run date automatically.
        </p>
      )}

      <div className="flex gap-3 flex-wrap">
        <Button
          size="sm"
          variant="destructive"
          disabled={seedRunning || clearRunning}
          onClick={async () => {
            setSeedRunning(true);
            setSeedLog([]);
            const result = await seedTestData((msg) =>
              setSeedLog((prev) => [...prev, msg])
            );
            if (!result.success) {
              setSeedLog((prev) => [...prev, `✗ Error: ${result.error}`]);
```

```
          toast.error("Seed failed — see log below");
        } else {
          toast.success("50 test users seeded successfully");
        }
        setSeedRunning(false);
      }}
    >
      {seedRunning ? "Seeding…" : "Seed 50 Test Users"}
    </Button>

    <Button
      size="sm"
      variant="outline"
      className="border-destructive text-destructive hover:bg-destructive hover:text-destructive-foreground"
      disabled={seedRunning || clearRunning}
      onClick={async () => {
        setClearRunning(true);
        setSeedLog([]);
        const result = await clearTestData((msg) =>
          setSeedLog((prev) => [...prev, msg])
        );
        if (!result.success) {
          setSeedLog((prev) => [...prev, `✗ Error: ${result.error}`]);
          toast.error("Clear failed — see log below");
        } else {
          queryClient.invalidateQueries({ queryKey: ["admin-groups"] });
          toast.success("Test data cleared");
        }
        setClearRunning(false);
      }}
    >
      {clearRunning ? "Clearing…" : "Clear Test Data"}
    </Button>
  </div>

  {/* Progress log */}
  {seedLog.length > 0 && (
    <div className="rounded-sm bg-black/80 text-green-400 font-mono text-xs p-3 space-y-0.5 max-h-40 overflow-y-auto
      {seedLog.map((line, i) => (
        <p key={i}>{line}</p>
      ))}
    </div>
  )}
 </div>
)}
</div>
```

Step 5 — Database: Allow Test Bookings Without Stripe

The `bookings` table likely has a `NOT NULL` constraint on `stripe_payment_intent_id`. Test bookings bypass Stripe, so either:

Option A (preferred): Check whether `stripe_payment_intent_id` is nullable in the existing schema. If it already allows `NULL`, no migration is needed.

Option B: If it is `NOT NULL`, add this migration:

Create file: `supabase/migrations/[timestamp]_allow_test_bookings.sql`

```sql
-- Allow bookings without a Stripe payment intent (for test/seed data only)
ALTER TABLE public.bookings
  ALTER COLUMN stripe_payment_intent_id DROP NOT NULL;
```

> **Note for Lovable:** Check the existing bookings table schema first. Only apply Option B if the column is currently `NOT NULL`.

Step 6 — End-to-End Test Flow (Instructions for the developer)

Once the seed tool is built, follow these steps to test the full matching flow:

1. **Open Admin → Groups**
2. Expand **Developer Tools** and click **"Seed 50 Test Users"**
3. Wait for the log to show "✓ 50 test users created and booked"
4. In the **Filter by Run Date** dropdown, select the run date the seeder used
5. Click **"Run Matching"** — this manually triggers the `match-groups` edge function
6. Verify the matching banner shows "Matched X groups" and the Pending tab populates with approximately 6–8 groups of 6–8 members each
7. Optionally edit a group (add/remove a member) to test admin editing
8. Click **"Approve All"**
9. Verify the groups move to the Approved tab
10. Log into one of the test accounts (email: `testuser+1@crew-test.dev`, password: `CrewTest2026!`) and navigate to **My Run**
11. Confirm the Crew, Route, and Chat tabs are visible with correct data
12. Open a second test account in a different browser and send a message — confirm it appears in real time in the first browser
13. Return to admin and click **"Clear Test Data"** to reset

Summary of All New Files

File	Action
`src/hooks/use-seed-test-data.ts`	CREATE
`supabase/functions/seed-test-data/index.ts`	CREATE
`supabase/migrations/[ts]_allow_test_bookings.sql`	CREATE (only if needed — see Step 5)
`src/pages/admin/Groups.tsx`	MODIFY (append Developer Tools section only)
`supabase/config.toml`	MODIFY (add seed-test-data function entry only)

Summary of Unchanged Files

Everything else — all auth, user-facing pages, onboarding, booking flow, existing edge functions, and UI components remain untouched.